HARP-PAF TECHNICAL SPECIFICATION

System Architecture & Algorithmic Foundations

Phase-Aware Adaptive Fusion for Hyper-Thread Resource Pairing and Single-Code Compilation Concurrency

Abstract: This document details the end-to-end technical architecture, mathematical formulations, and operational algorithms of the HARP-PAF research platform. It covers both the **Multi-Workload SMT Scheduling Engine** (using Intel EMON hardware PMU counters, interference stressor probing, Bayesian adaptive feature fusion, bottleneck interaction regression, and dynamic co-scheduling) and the **Single Code Optimization Engine** (Tree-sitter AST translation-unit decomposition, parallel multi-threaded build coordination, and pairwise build contention correlation).

Platform Version	Target Processor	Execution Engine	Compiler Stack
HARP-PAF v0.3.0	Intel Core i5-1235U (Alder Lake)	FastAPI + Vite React 19	GCC / G++ 14.2.0 (MinGW-w64)

1. Problem Formulation & Hardware Context

Modern superscalar microprocessors employ **Simultaneous Multi-Threading (SMT)**—known commercially as Intel Hyper-Threading—to maximize execution pipeline utilization. An SMT-enabled physical core exposes two or more logical CPUs (hardware threads) that dynamically share critical on-chip execution resources:

- **Execution Ports & Functional Units:** Integer ALUs (Port 0, 1, 5, 6), Vector/FP units and dividers (Port 0, 1, 5).
- **Memory Hierarchy:** Shared Level-1 Data Cache (L1D, 48 KB), Level-2 Cache (L2, 1.25 MB per P-core), Load/Store execution buffers, and Miss Status Holding Registers (MSHRs).
- **Frontend & Prediction:** Branch Target Buffers (BTBs), Reorder Buffer (ROB) partitions, and Instruction Decoders.

When two threads requiring the same bottleneck resource are co-scheduled on sibling SMT threads, destructive resource contention occurs, resulting in severe slowdowns (often exceeding 40%), memory stall bubbles, and cache thrashing. Conversely, complementary workloads (e.g., an Integer ALU compute thread paired with a memory-bandwidth-bound streaming thread) achieve near-linear aggregate throughput gains.

The HARP-PAF Objective: Given a set $W = \{w_1, w_2, \dots, w_n\}$ of approved independent C++ workloads and a physical core with two SMT sibling threads, find a non-preemptive schedule that minimizes total **makespan**:

> $$\text{Makespan}(W) = \max_{w_i \in W} \left(t_{\text{finish}}(w_i) \right) - \min_{w_i \in W} \left(t_{\text{start}}(w_i) \right)$$

---

2. End-to-End System Architecture

The application follows a decoupled client-server architecture consisting of a high-performance Python FastAPI backend and an interactive, real-time React 19 frontend:

System Subsystems Overview:

1. **Frontend Web Tier (React 19 + Vite):** Single-Page Application (SPA) providing real-time scheduling Gantt charts, interactive correlation scatter plots, hardware contention radar charts, and source-code splitting diagnostics.
2. **API & Process Dispatcher (FastAPI + Uvicorn):** RESTful API handling workload upload, compilation verification, isolated execution orchestration, and single-code AST splitting.
3. **Hardware Telemetry & Topology Engine:** Detects physical core layout and SMT sibling thread pairs via Windows `GetLogicalProcessorInformationEx` and Linux `/sys/devices/system/cpu`. Pinned execution guarantees measurement drift elimination.
4. **Workload Characterization Engine:** Combines GNU `objdump` static disassembly analysis with Intel EMON hardware counter sampling (100 ms phase windows) and 5 resource-specific stressor probes.
5. **Predictive Slowdown & Pairing Engine:** Evaluates pairwise interference using a calibrated Bottleneck Sensitivity × Aggressiveness Non-Negative Least Squares (NNLS) model.
6. **Dynamic SMT Co-Scheduler:** Online event-driven dispatch algorithm executing on P-Core 0 (Logical CPUs 0 and 1) while isolating measurement harness threads on E-Core 11.
7. **Single-Code Modularizer & Parallel Compiler:** Tree-sitter AST parser that decomposes monolithic C++ programs into modular translation units and orchestrates concurrent compilation across worker threads.

Hardware Core Topology Mapping

On hybrid heterogeneous architectures (e.g., Intel 12th Gen Alder Lake i5-1235U), SMT is only available on Golden Cove Performance Cores (P-cores). Gracemont Efficient Cores (E-cores) are single-threaded. HARP-PAF maps execution as follows:

CPU Identifier	Microarchitecture	Role in HARP-PAF Platform
Logical CPU 0	Golden Cove (P-Core 0, SMT Thread 0)	Primary Workload Co-Execution Thread
Logical CPU 1	Golden Cove (P-Core 0, SMT Thread 1)	Sibling Workload Co-Execution Thread
Logical CPU 2–3	Golden Cove (P-Core 1, SMT Thread 0/1)	Secondary Evaluation / Build Core
Logical CPU 4–10	Gracemont (E-Cores 0–6)	System OS Background Tasks
Logical CPU 11	Gracemont (E-Core 7)	**Dedicated Measurement Harness & Controller (Pinned)**

---

3. Mathematical Formulations & Predictive Models

3.1 Adaptive Bayesian Feature Fusion (Equation F)

Static disassembly analysis cannot observe cache miss rates or runtime branch mispredictions, while runtime hardware sampling suffers from cold-start transient noise. HARP-PAF computes a fused feature vector $x(i, k, t)$ for workload i, feature k, and phase t:

$$x(i, k, t) = q(i, k, t) \cdot d(i, k, t) + (1 - q(i, k, t)) \cdot s(i, k)$$

where $s(i, k)$ is the normalized static binary estimate, $d(i, k, t)$ is the runtime counter estimate, and q is the dynamic confidence weight:

$$q(i, k, t) = Q_{\max} \cdot \max\left(0, 1 - \frac{\sigma}{\sigma_{\text{ref}}}\right) \cdot \min\left(1, \frac{n_t}{n_{\min}}\right)$$

Parameters: $Q_{\max} = 0.90$, $\sigma_{\text{ref}} = 0.25$, $n_{\min} = 2$ windows (100 ms per window). If hardware counters are unavailable, $q = 0$.

3.2 Interference Sensitivity Probing (Equation S1)

Workload sensitivity to each shared resource $k \in \{\text{int}, \text{fp}, \text{stream}, \text{cache}, \text{branch}\}$ is measured empirically by co-running workload i against five tuned micro-stressors:

$$\text{sens}(i, k) = \min\left(2.0, \; \frac{T(i \mid \text{stressor } k)}{T_{\text{solo}}(i)} - 1\right)$$

3.3 Hardware Aggressiveness Signature (Equation S2)

The pressure a co-runner generates on resource k during phase t is normalized against the stressor's calibrated reference signature g_k:

$$\text{aggr}(j, k, t) = \min\left(1.5, \; \frac{x(j, \text{agg}_k, t)}{g_k}\right)$$

- **Integer ALU:** $\min\left(1, \frac{\text{IPC} \cdot m_{\text{int}}}{5}\right)$ (5 ALU execution ports).
- **FP / Vector:** $\max\left(\text{divider_busy}, \min\left(1, \frac{\text{IPC} \cdot m_{\text{fp}}}{3}\right)\right)$ (3 vector ports).
- **Memory Bandwidth:** $\frac{\text{LLC_misses} \times 64\,\text{B}}{\Delta t \times B_{\text{socket}}}$ ($B_{\text{socket}} = 20\,\text{GB/s}$).
- **Cache Pressure:** $\text{footprint} \cdot (1 - \text{LLC_miss_ratio})$, where $\text{footprint} = \frac{\text{LLC_refs} / 1000\,\text{instr}}{40}$.
- **Branch Divergence:** $\frac{\text{branch_misses} / 1000\,\text{instr}}{40}$.

3.4 Bottleneck Contention Interaction Kernel (Equation S3)

To avoid double-counting overlapping contention, HARP-PAF applies the **Bottleneck Dominance Principle**: a thread's pipeline is stalled predominantly by its single most contended resource:

```
$$H(i, j) = \sum_t \pi_t \cdot \frac{1}{2} \left[ \max_k \left( \text{sens}(i, k) \cdot \text{aggr}(j, k, t) \right) + \max_k \left( \text{sens}(j, k) \cdot \text{aggr}(i, k, t) \right) \right]$$
```

3.5 Calibrated Slowdown Model (Equation S4)

Pairwise slowdown $\hat{y}(i, j)$ is predicted via Non-Negative Least Squares (NNLS) regression fitted on held-out calibration pairs:

```
$$\hat{y}(i, j) = \max\left(0, \; b_0 + b \cdot H(i, j)\right)$$
```

Calibrated parameters: $b_0 = 0.0381$, $b = 0.7003$. This model achieves a held-out Mean Absolute Error (MAE) of **0.069** and a rank correlation of **0.84** (compared with 0.129 MAE and 0.33 correlation for earlier linear formulations).

---

4. Operational Algorithms

Algorithm 1: Workload Characterization Pipeline

```
Input: Workload source file w (.cpp), target compiler GCC
Output: Fused characterization tuple {T_solo, sens, phases, x_fused}

Compile w with: g++ -O2 -std=c++17 w.cpp -o w.exe
Execute w.exe solo; verify stdout matches expected SHA-256 manifest
Warm up CPU for 10 seconds to stabilize core thermal state
Measure T_solo across 3 iterations (take median)
Run w.exe with Intel EMON; capture 100 ms counter windows
Detect phase transitions when window metrics deviate by > 0.25 from running mean
For each stressor k in {int_alu, fp_div, stream, cache, branch}:
    Pin w.exe to Logical CPU 0 and stressor_k to Logical CPU 1
    Measure T(w | stressor_k) and compute sens(w, k) via Equation S1
Disassemble w.exe with objdump; extract loop-weighted instruction mix
Compute fused features x(w, k, t) via Equation F
Return characterization tuple
```

Algorithm 2: Dynamic SMT Co-Scheduling (Online Dispatch)

```
Input: Queue of pending characterized workloads Q, Logical CPU 0, Logical CPU 1
Output: Execution makespan and Gantt co-run timeline

Pin measurement harness thread to E-core Logical CPU 11
Select w_first = argmax_{w in Q} (T_solo(w)); dispatch to Logical CPU 0
Select w_second = argmin_{w in Q} (predicted_slowdown(w_first, w)); dispatch to CPU 1
While Q is not empty or active threads running:
    Wait for hardware thread notification (workload w_finished terminates)
    Record completion time and output checksum
    Identify surviving co-runner w_active on sibling thread
    Estimate remaining execution progress p of w_active
    If Q is not empty:
        Select candidate c* in Q minimizing:
            Cost(c) = predicted_slowdown(w_active[p..1], c) + mu * max(0, worst_slowdown - tau) + nu * sigma
        Dispatch c* to the free hardware thread
Return makespan
```

Algorithm 3: Single-Code C++ AST Decomposition & Parallel Build Coordination

```
Input: Monolithic C++ source file S, worker count P = 4, output directory O
Output: Generated translation units, build benchmark results, Gantt schedule

1: Parse S into Concrete Syntax Tree (CST) using Tree-sitter CPP grammar
2: Identify root AST nodes: struct_specifiers, class_specifiers, preprocessor directives, functions
3: Extract common preamble and forward function declarations into generated.h
4: Split helper functions f_1, f_2, ..., f_m into module_01.cpp, module_02.cpp, ...
5: Place main() into main.cpp (including generated.h)
6: Original build: Compile monolithic S with g++ -O2 -o original.exe; record T_orig
7: Parallel build: Initialize ThreadPoolExecutor(max_workers = P)
8: For each translation unit u_i in parallel:
9:     Record start timestamp t_start and assign worker thread ID w_id
10:    Compile g++ -O2 -c u_i.cpp -o u_i.o; record end timestamp t_end
11: Link all object files: g++ u_1.o ... u_k.o -o split.exe; record T_link
12: Execute original.exe and split.exe; verify identical stdout outputs
13: Compute pairwise build contention matrix C(i, j) and correlation coefficient r
14: Generate persistent job artifacts (job.json, results.json)
15: Return benchmark, generated files, and schedule
```

---

5. Empirical Performance Comparison

Evaluation conducted on an Intel Core i5-1235U with 8 resource-specific C++ benchmark workloads over 10 repeated randomized trials demonstrates that HARP-PAF's phase-aware dynamic scheduling substantially outperforms standard OS scheduling heuristics:

Scheduling Strategy	Median Makespan	Speedup vs Random	Runtime Reduction	Throughput Gain	Trial Superiority
HARP-PAF (Dynamic)	**3.20 s**	**1.09×**	**8.4%**	**+9.1%**	**10 / 10 runs**
HARP-PAF (Static Pairs)	3.45 s	1.01×	1.2%	+1.3%	6 / 10 runs
Random Co-Scheduling	3.49 s	1.00×	0.0%	0.0%	Baseline
OS Default (Linux CFS / Windows)	3.53 s	0.99×	-1.1%	-1.1%	1 / 10 runs
PORT Contention-Aware	3.74 s	0.93×	-7.2%	-6.7%	0 / 10 runs
Serial Execution (No SMT)	5.47 s	0.64×	-56.7%	-36.2%	0 / 10 runs